

The logo consists of a white rounded square with the word "Kesh" in a bold, dark blue sans-serif font centered within it.

Kesh

Documentation technique

Manuel administrateur

Installation, configuration et exploitation de Kesh dans un
environnement de production

Version **0.1.0-dev** | 2026-05-20

Cible release : **v0.1**

<https://github.com/guycorbaz/kesh>

Table des matières

1	À propos de ce manuel	5
1.1	Public cible	5
1.2	Pré-requis de lecture	5
1.3	Conventions typographiques	5
2	Présentation de Kesh	6
2.1	Objectif du logiciel	6
2.2	Public cible du produit	6
2.3	Architecture technique haute	7
2.4	Licence	7
3	Prérequis système	7
3.1	Système d'exploitation	7
3.2	Logiciels requis	7
3.3	Ressources matérielles minimales	8
3.4	Ports réseau	8
3.4.1	Changer le port d'écoute (conflit port 80, ex. Synology DSM Web Station)	8
4	Installation	9
4.1	Méthode recommandée : Docker Compose	9
4.1.1	Étape 1 — Préparer l'environnement	9
4.1.2	Étape 2 — Récupérer le <code>docker-compose.yml</code>	10
4.1.3	Étape 3 — Créer le fichier <code>.env</code>	10
4.1.4	Étape 4 — Démarrer les services	10
4.1.5	Étape 5 — Vérifier l'installation	11
4.2	Configuration du reverse proxy HTTPS	11
4.3	Alternative : déploiement via Caddy	12
4.4	Alternative : déploiement via Traefik (avec firewall applicatif)	12
4.4.1	Compose Traefik en sidecar	13
4.4.2	Middlewares firewall applicatif	14
4.4.3	Plugins WAF avancés (optionnel, recommandé Internet-exposed)	15
4.4.4	Considérations sécurité spécifiques	15
4.5	Alternative : déploiement sur Synology DSM	16
4.5.1	Prérequis Synology DSM	16
4.5.2	Étape 1 — Préparer la structure de répertoires	16
4.5.3	Étape 2 — Récupérer <code>docker-compose.yml</code> et <code>.env</code>	17
4.5.4	Étape 3 — Importer le projet dans Container Manager	17
4.5.5	Étape 4 — Configurer le reverse proxy DSM (Application Portal)	18

4.5.6	Étape 5 — Vérification post-installation	19
4.5.7	Limitations Container Manager DSM	19
5	Configuration	19
5.1	Variables d'environnement complètes	19
5.1.1	Variables obligatoires	20
5.1.2	Variables optionnelles avec valeurs par défaut	21
5.1.3	Variables liées au monitoring (optionnelles)	21
5.2	Configuration des logs fichier	22
5.3	Configuration MariaDB	22
5.3.1	Paramètres recommandés	23
5.3.2	Initialisation manuelle de la base	23
5.4	Configuration multi-tenant	24
5.4.1	Concept de <i>company</i>	24
5.4.2	Isolation des données	24
5.4.3	Cas d'usage fiduciaire	24
6	Premier démarrage	25
6.1	Création du compte administrateur initial (onboarding self-service)	25
6.1.1	Procédure (nouveau flow recommandé v0.1.2+)	25
6.1.2	Procédure alternative — bootstrap déclaratif via <code>.env</code>	25
6.2	J'ai oublié mon mot de passe administrateur (recovery break-glass)	26
6.2.1	Procédure step-by-step	26
6.2.2	Matrice de comportement au boot — vue d'ensemble	27
6.3	Déploiement LAN HTTP-only (sans HTTPS) — v0.1.3+	27
6.3.1	Décision : HTTPS ou HTTP-only ?	28
6.3.2	Procédure HTTP-only (<code>KESH_COOKIE_SECURE=false</code>)	28
6.3.3	Valeurs acceptées et fail-fast	29
6.4	Configuration de la company via l'onboarding	29
6.5	Choix du plan comptable	30
6.6	Configuration des exercices comptables	30
6.7	Création des utilisateurs et rôles	30
7	Sauvegarde et restauration	31
7.1	Importance de la sauvegarde	31
7.2	Stratégie recommandée	31
7.3	Script de sauvegarde mariadb-dump	32
7.4	Stockage offsite	33
7.5	Restauration depuis sauvegarde	33
7.6	Test de restauration périodique (OLiCo Art. 10 al. 1)	33
7.7	Backup natif sur Synology DSM (Hyper Backup + Snapshot Replication)	34
7.7.1	Hyper Backup — backup incrémental + offsite intégré	34
7.7.2	Snapshot Replication — recovery instantané point-in-time	35

7.7.3	Stratégie 3-2-1 sur DSM	36
8	Mise à jour de Kesh	37
8.1	Process de release Kesh	37
8.2	Suivre les notifications de release	37
8.3	Procédure de mise à jour standard	37
8.4	Mise à jour majeure (changement de version X)	38
8.5	Rollback en cas d'échec	38
9	Sécurité	38
9.1	HTTPS obligatoire en production	39
9.2	Gestion des secrets	39
9.3	Authentification JWT + refresh tokens	39
9.4	Clés API (PAT) — accès programmatique	39
9.5	RBAC et permissions	40
9.6	Audit-trail (audit_log)	40
9.7	Conformité OLICo Art. 9 — intégrité des supports modifiables	41
9.8	Sécurité réseau (firewall)	41
9.9	Mises à jour de sécurité du système hôte	41
10	Monitoring et logs	42
10.1	Logs applicatifs	42
10.2	Centralisation des logs	42
10.3	Métriques Prometheus	42
10.4	Alerting recommandé	43
11	Conformité légale suisse	43
11.1	Code des obligations Art. 957a — Tenue de la comptabilité	43
11.2	Code des obligations Art. 958f — Conservation 10 ans	44
11.3	Ordonnance OLICo (RS 221.431)	44
11.4	LSCSE (signature électronique qualifiée)	44
11.5	Loi sur la TVA (LTVA)	44
11.6	Documentation à conserver	44
12	Dépannage	45
12.1	Kesh ne démarre pas	45
12.2	Erreurs 401 en cascade côté frontend	45
12.3	Performance lente sur de gros datasets	46
12.4	Reset du mot de passe administrateur	46
12.5	Données corrompues détectées	46

13 Maintenance	47
13.1 Rotation des logs	47
13.2 Maintenance MariaDB	47
13.3 Vérification périodique de l'intégrité (OLiCo Art. 10)	47
14 Annexes	48
14.1 Liste complète des variables d'environnement	48
14.2 Commandes CLI utiles	48
14.3 Référence des ports	49
14.4 Glossaire	49
14.5 Liens externes	49
14.6 Support et communauté	50

1 À propos de ce manuel

Ce manuel s'adresse aux administrateurs système, responsables DevOps et fiduciaires souhaitant déployer et exploiter Kesh dans un environnement de production. Il couvre l'ensemble du cycle de vie opérationnel d'une instance Kesh : installation, configuration, sauvegarde, mise à jour, sécurité, conformité légale suisse et dépannage.

1.1 Public cible

Ce document est rédigé pour :

- Les administrateurs système (Linux ou macOS) responsables de l'hébergement d'une instance Kesh pour une ou plusieurs PME.
- Les fiduciaires souhaitant proposer Kesh comme solution comptable à leurs clients en mode self-hosting.
- Les développeurs internes d'une PME prenant en charge l'auto-hébergement de leur instance.
- Les responsables sécurité ou conformité devant valider la posture Kesh face aux exigences du Code des obligations suisse (CO Art. 957a, 958f) et de l'Ordonnance OLICo (RS 221.431).

1.2 Pré-requis de lecture

Le lecteur est supposé familier avec :

- Les concepts de base de Docker et Docker Compose (images, volumes, networks).
- L'administration d'un système Linux (CLI, systemd, gestion de paquets, permissions de fichiers).
- Les principes d'une base de données relationnelle (MariaDB/MySQL).
- Les concepts de reverse proxy HTTPS (Nginx, Traefik ou équivalent).
- Les notions de base de cryptographie appliquée (TLS, certificats, secrets).

1.3 Conventions typographiques

- `commande shell` : commande à exécuter dans un terminal.
- `/chemin/fichier` : chemin vers un fichier ou répertoire sur le système.
- `Ctrl+S` : combinaison de touches clavier.
- Les boîtes colorées : *Note* (information), *Astuce*, *Attention* (avertissement), *Danger* (risque critique).

i Note

Ce manuel décrit Kesh version 0.1.0-dev. Pour la dernière version, consultez <https://github.com/guycorbaz/kesh>. Voir la section finale « À propos de cette version » pour les modalités de mise à jour.

2 Présentation de Kesh

2.1 Objectif du logiciel

Kesh est un **logiciel de comptabilité et de gestion** destiné aux indépendants, petites et moyennes entreprises (PME) et associations en Suisse. Il fournit un système complet conforme aux exigences du Code des obligations suisse, incluant :

- **Comptabilité en partie double** avec plan comptable suisse PME (Sterchi, KMU, ou personnalisé).
- **Facturation QR Bill 2.2** conforme au standard SIX Interbank Clearing (norme suisse 2020+).
- **Import bancaire** CAMT.053 (ISO 20022) et CSV multi-encodage avec profils banque réutilisables.
- **Réconciliation automatique et manuelle** des transactions bancaires avec règles d'affectation.
- **Rapports comptables** : bilan, compte de résultat, balance, journal.
- **Exports** légaux : PDF par rapport, ZIP global de souveraineté avec SHA-256 d'intégrité.
- **Multi-utilisateurs** avec contrôle d'accès basé sur les rôles (RBAC).
- **Multi-tenant** : isolation stricte des données par société (*company_id*).
- **Multilingue** : interface utilisateur en français, allemand, italien et anglais.

2.2 Public cible du produit

Kesh est conçu pour :

- Les **entreprises individuelles** et les **sociétés de personnes** avec un chiffre d'affaires supérieur à CHF 500'000 (régime « comptabilité complète » selon CO Art. 957 al. 1).
- Les **entreprises individuelles** et **sociétés de personnes** avec un chiffre d'affaires inférieur à CHF 500'000 (régime « recettes-dépenses + patrimoine » selon CO Art. 957 al. 2).
- Les **personnes morales** (SA, Sàrl, coopératives, associations, fondations).
- Les **fiduciaires** gérant plusieurs dossiers clients via le mode multi-tenant.

Kesh n'est pas conçu pour les grands groupes soumis à l'audit ordinaire (Art. 727 CO, seuils 20/40/250 selon Art. 727 al. 1 ch. 2 CO) ni aux entités cotées en bourse soumises à IFRS / Swiss GAAP RPC obligatoires.

2.3 Architecture technique haute

- **Backend** : Rust 1.85+ (édition 2024) avec le framework HTTP Axum et le client SQL SQLx.
- **Frontend** : SvelteKit 2 avec Svelte 5 et TypeScript, bundler Vite, CSS via Tailwind 4.
- **Base de données** : MariaDB 11.4 (compatibilité MySQL 8 maintenue, mais MariaDB recommandée pour les optimisations spécifiques utilisées).
- **Déploiement** : Docker Compose, image officielle [gcorbaz/kesh:latest](#) sur Docker Hub.
- **Authentification** : JWT (HS256) avec refresh tokens et rotation, durée d'expiration courte (15 min) compensée par rafraîchissement automatique côté frontend.
- **Multi-tenant** : claim `company_id` dans le JWT et scoping strict sur toutes les requêtes API et toutes les requêtes SQL (cf. Story Epic 7-1 « audit multi-tenant scoping refactor »).

2.4 Licence

Kesh est distribué sous [licence EUPL 1.2](#) (European Union Public Licence). Cette licence est compatible GPL et permet l'usage commercial, la modification et la redistribution sous réserve de conserver la licence et les mentions d'attribution.

3 Prérequis système

3.1 Système d'exploitation

Kesh est officiellement supporté sur :

- **Linux** : Debian 12+, Ubuntu 22.04 LTS+, Fedora 38+, RHEL 9+, Arch Linux (rolling).
- **macOS** : 13 Ventura+ (architecture Intel ou Apple Silicon).

Le déploiement est officiellement supporté sur Linux uniquement. macOS est utilisable pour le développement et l'évaluation, mais non recommandé en production.

Le déploiement sur Windows (avec WSL2) est possible mais non officiellement testé.

3.2 Logiciels requis

- **Docker** ≥ 24.0 (Docker Engine ou Docker Desktop).
- **Docker Compose** ≥ 2.20 (plugin officiel [docker compose](#)).
- Un **reverse proxy HTTPS** en production (Nginx, Caddy, Traefik, HAProxy, ou équivalent).
- Un système de **sauvegarde MariaDB** (mariadb-dump cron, mariabackup, ou solution tierce).
- Optionnel : **Git** pour cloner le dépôt et suivre les mises à jour.

3.3 Ressources matérielles minimales

Ressource	Minimum	Recommandé
CPU	1 vCPU	2–4 vCPU
RAM	1 Go	4 Go
Stockage	5 Go (DB + logs)	50 Go (conservation 10 ans incluse)
Réseau	1 Mbps stable	10 Mbps + IPv4 publique
Disque	SSD recommandé pour DB	SSD obligatoire pour DB

i Note

Le stockage est dominé par l'*audit_log* et les pièces justificatives. Comptez environ **50–200 Mo par exercice** pour une PME standard. Pour une conservation OLICo 10 ans, prévoyez **1–3 Go** en moyenne.

3.4 Ports réseau

Port	Protocole	Usage	Exposition
443	HTTPS	Accès utilisateur via reverse proxy	Publique (Internet)
80	HTTP	API Axum backend (interne)	Localhost uniquement
5173	HTTP	SvelteKit dev server (dev seul)	Localhost uniquement
3306	MySQL/MariaDB	Connexion base de données	Localhost ou réseau privé

! Attention

Ne jamais exposer le port 80 (API Axum) ni le port 3306 (MariaDB) directement sur Internet. Toujours passer par un reverse proxy HTTPS qui termine TLS et fait suivre vers `localhost`.

3.4.1 Changer le port d'écoute (conflit port 80, ex. Synology DSM Web Station)

Le port `80` par défaut peut entrer en conflit avec un service existant sur l'hôte — typiquement **Synology DSM Web Station** qui occupe également ce port, ou un `nginx` local, un `IIS`, etc. Quatre options d'override existent, selon votre contexte :

1. **Garder le port applicatif 80 mais remapper côté host** (recommandé) — dans `docker-compose.prod, dev.` changer le mapping `"127.0.0.1:80:80"` vers `"127.0.0.1:8080:80"` (ou tout autre `HOST_PORT` libre). L'application reste sur 80 côté container ; URL d'accès : `http://localhost:8080`.
2. **Changer le port applicatif lui-même** — dans `.env`, décommenter et adapter :

```
KESH_PORT=3000
```

puis aligner le mapping `docker-compose.*.yaml` sur `"127.0.0.1:3000:3000"`. URL d'accès : `http://localhost:3000`.

3. **Container sur une IP dédiée (Docker macvlan/ipvlan)** — configurer le container avec sa propre IP via réseau `macvlan` ; le mapping host devient inutile (le container répond directement sur son `IP:80`). Procédure spécifique à votre infrastructure réseau.
4. **Mode `cargo run` natif (hors Docker) sur Linux non-root** — Linux n'autorise pas les utilisateurs non-root à bind un port `<1024` (*security feature*). Dans ce contexte, set **obligatoirement** `KESH_PORT=3000` (ou un autre port `≥ 1024`) dans votre `.env` local. Alternatives possibles mais déconseillées : lancer avec `sudo` , ou ajouter la *capability* via `sudo setcap 'cap_net_bind_service=+`

! Astuce

Sur Synology DSM, le port 80 est par défaut redirigé vers DSM Web Station. Sans override, le container Kesh ne pourra pas démarrer («port already in use»). La méthode (1) «remap côté host» est la plus simple pour cohabiter — par exemple `"127.0.0.1:8080:80"` + reverse proxy DSM vers `localhost:8080`.

4 Installation

4.1 Méthode recommandée : Docker Compose

Kesh est livré sous forme d'image Docker. Le déploiement standard utilise Docker Compose.

4.1.1 Étape 1 — Préparer l'environnement

Créer un répertoire dédié pour l'instance Kesh :

```
sudo mkdir -p /opt/kesh
sudo chown $USER:$USER /opt/kesh
cd /opt/kesh
```

4.1.2 Étape 2 — Récupérer le `docker-compose.yml`

Télécharger le fichier de référence depuis le dépôt officiel :

```
curl -fsSL
  https://raw.githubusercontent.com/guycorbaz/kesh/main/docker-compose.yml \
  -o docker-compose.yml
```

4.1.3 Étape 3 — Créer le fichier `.env`

Copier le fichier d'exemple et l'adapter :

```
curl -fsSL
  https://raw.githubusercontent.com/guycorbaz/kesh/main/.env.example \
  -o .env
```

Éditer `.env` et renseigner au minimum les variables suivantes (cf. section 5.1 pour la liste complète) :

```
# Secret JWT - générer avec : openssl rand -hex 64
JWT_SECRET=<long secret aleatoire 128+ caracteres>

# Mot de passe DB
MARIADB_ROOT_PASSWORD=<mot de passe fort>
MARIADB_PASSWORD=<mot de passe utilisateur applicatif>
MARIADB_DATABASE=kesh
MARIADB_USER=kesh

# URL de connexion DB (composée à partir des variables ci-dessus)
DATABASE_URL=mysql://kesh:<MARIADB_PASSWORD>@db:3306/kesh

# URL publique (pour les liens dans emails, QR Bills, etc.)
KESH_PUBLIC_URL=https://compta.exemple.ch
```

X Danger

Ne jamais committer le fichier `.env` dans un dépôt Git. Il contient les secrets de production. Ajoutez-le à `.gitignore`.

4.1.4 Étape 4 — Démarrer les services

```
docker compose up -d
```

Vérifier que les services sont démarrés :

```
docker compose ps
docker compose logs -f kesh
```

Au premier démarrage, Kesh exécute automatiquement les migrations de schéma DB. Cela peut prendre 10–30 secondes selon la machine.

4.1.5 Étape 5 — Vérifier l'installation

Tester l'accès à l'API :

```
curl http://localhost/health
# Attendu : {"status":"ok","version":"<keshVersion>"}
```

4.2 Configuration du reverse proxy HTTPS

En production, Kesh **doit** être accessible via HTTPS uniquement. Voici un exemple minimal avec Nginx :

```
# /etc/nginx/sites-available/kesh
server {
    listen 443 ssl http2;
    server_name compta.exemple.ch;

    ssl_certificate
    /etc/letsencrypt/live/compta.exemple.ch/fullchain.pem;
    ssl_certificate_key /etc/letsencrypt/live/compta.exemple.ch/privkey.pem;

    # Strong cipher suite
    ssl_protocols TLSv1.2 TLSv1.3;
    ssl_prefer_server_ciphers on;

    # Headers de securite
    add_header Strict-Transport-Security "max-age=31536000;
    includeSubDomains" always;
    add_header X-Content-Type-Options "nosniff" always;
    add_header X-Frame-Options "DENY" always;
    add_header Referrer-Policy "strict-origin-when-cross-origin" always;

    location / {
        proxy_pass http://localhost;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto https;

        # Pour les uploads de pieces comptables et imports CAMT.053
        client_max_body_size 50M;

        # Timeouts adaptes pour les operations longues (exports ZIP)
        proxy_read_timeout 300s;
    }
}
```

```
    }  
}  
  
server {  
    listen 80;  
    server_name compta.exemple.ch;  
    return 301 https://$host$request_uri;  
}
```

Activer la configuration :

```
sudo ln -s /etc/nginx/sites-available/kesh /etc/nginx/sites-enabled/  
sudo nginx -t && sudo systemctl reload nginx
```

4.3 Alternative : déploiement via Caddy

Caddy gère automatiquement les certificats Let's Encrypt et représente une alternative plus simple :

```
# /etc/caddy/Caddyfile  
compta.exemple.ch {  
    reverse_proxy localhost {  
        header_up X-Forwarded-Proto https  
    }  
    encode gzip  
    header {  
        Strict-Transport-Security "max-age=31536000; includeSubDomains"  
        X-Content-Type-Options "nosniff"  
        X-Frame-Options "DENY"  
    }  
}
```

4.4 Alternative : déploiement via Traefik (avec firewall applicatif)

Traefik est particulièrement adapté quand Kesh est exposé sur Internet : il intègre nativement des middlewares de *firewall applicatif* (rate limiting, IP whitelist, headers de sécurité) et supporte des plugins communautaires plus avancés (CrowdSec pour la détection d'intrusion, ModSecurity-CRS pour des règles WAF type OWASP). La configuration ci-dessous suppose Traefik 3.x déployé en sidecar dans le même Docker Compose que Kesh.

4.4.1 Compose Traefik en sidecar

Ajouter le service Traefik à votre `docker-compose.yml` (ou utiliser un `docker-compose.override.yml` dédié) :

```
services:
  traefik:
    image: traefik:v3.2
    container_name: kesh-traefik
    restart: unless-stopped
    command:
      # Entrypoints HTTP/HTTPS (HTTP redirige vers HTTPS).
      - --entrypoints.web.address=:80
      - --entrypoints.web.http.redirections.entryPoint.to=websecure
      - --entrypoints.web.http.redirections.entryPoint.scheme=https
      - --entrypoints.websecure.address=:443
      # Certificats Let's Encrypt automatiques (challenge HTTP-01).
      - --certificatesresolvers.le.acme.email=admin@exemple.ch
      - --certificatesresolvers.le.acme.storage=/letsencrypt/acme.json
      - --certificatesresolvers.le.acme.httpchallenge.entrypoint=web
      # Provider Docker - lit les labels sur les autres services.
      - --providers.docker=true
      - --providers.docker.exposedbydefault=false
      # Logs format JSON pour ingestion ultérieure (Loki, ELK, etc.).
      - --accesslog=true
      - --accesslog.format=json
      - --log.level=INFO
    ports:
      - "80:80"
      - "443:443"
    volumes:
      - /var/run/docker.sock:/var/run/docker.sock:ro
      - traefik-letsencrypt:/letsencrypt
    networks:
      - kesh-internal

  kesh-api:
    # ... votre config kesh-api existante ...
    # NE PAS publier le port 80 sur l'hôte (Traefik y accède via réseau Docker).
    expose:
      - "80"
    networks:
      - kesh-internal
    labels:
      - traefik.enable=true
      # Router HTTPS + cert Let's Encrypt.
      - traefik.http.routers.kesh.rule=Host(`compta.exemple.ch`)
```

```

- traefik.http.routers.kesh.entrypoints=websecure
- traefik.http.routers.kesh.tls.certresolver=le
- traefik.http.services.kesh.loadbalancer.server.port=80
# Chaîne de middlewares firewall applicatif (cf. ci-dessous).
-
traefik.http.routers.kesh.middlewares=kesh-ratelimit@docker,kesh-headers@docker

volumes:
  traefik-letsencrypt:

networks:
  kesh-internal:
    driver: bridge

```

4.4.2 Middlewares firewall applicatif

Définir les middlewares via labels sur le service `kesh-api` (ou dans un `dynamic.yml` dédié) :

```

labels:
  # ... labels routeur ci-dessus ...

  # 1. Rate limiting global - défense contre brute-force et scraping.
  #   Limite moyenne 50 req/s par IP, burst 100. Défense en
  #   profondeur du rate-limit applicatif Kesh (KESH_RATE_LIMIT_*).
  - traefik.http.middlewares.kesh-ratelimit.ratelimit.average=50
  - traefik.http.middlewares.kesh-ratelimit.ratelimit.burst=100
  -
traefik.http.middlewares.kesh-ratelimit.ratelimit.sourcecriteria.ipstrategy.depth=1

  # 2. Headers de sécurité (durcissement OWASP).
  - traefik.http.middlewares.kesh-headers.headers.stsSeconds=31536000
  -
traefik.http.middlewares.kesh-headers.headers.stsIncludeSubdomains=true
  - traefik.http.middlewares.kesh-headers.headers.stsPreload=true
  - traefik.http.middlewares.kesh-headers.headers.contentTypeNosniff=true
  - traefik.http.middlewares.kesh-headers.headers.frameddeny=true
  -
traefik.http.middlewares.kesh-headers.headers.referrerPolicy=strict-origin-when-cross-origin
  -
traefik.http.middlewares.kesh-headers.headers.permissionsPolicy=geolocation=(),microphone=()

  # 3. (OPTIONNEL) IP whitelist pour endpoints sensibles - exemple
  #   pour bloquer /api/v1/_test/* (qui n'existe qu'en KESH_TEST_MODE
  #   mais on défend en profondeur). À adapter à vos IPs admin.
  # -
traefik.http.middlewares.kesh-admin-only.ipallowlist.sourcerange=192.168.1.0/24,2001:db8:

```

4.4.3 Plugins WAF avancés (optionnel, recommandé Internet-exposed)

Pour un firewall applicatif complet face à Internet, deux plugins Traefik communautaires sont éprouvés :

CrowdSec — détection d'intrusion collaborative CrowdSec analyse les logs Traefik en temps réel, détecte les scans/scrapes/brute-force, et partage les IPs malveillantes avec une CTI communautaire (équivalent crowd-sourced de *Fail2Ban* + base de réputation IP globale).

```
# Activer le plugin dans la commande Traefik (ajouter dans le `command:`
  ci-dessus) :
-
  --experimental.plugins.crowdsec-bouncer.moduleName=github.com/maxlerebourg/crowdsec-bouncer
- --experimental.plugins.crowdsec-bouncer.version=v1.3.5

# Puis sur kesh-api, ajouter le middleware :
labels:
-
  traefik.http.middlewares.kesh-crowdsec.plugin.crowdsec-bouncer.enabled=true
-
  traefik.http.middlewares.kesh-crowdsec.plugin.crowdsec-bouncer.crowdsecApiKey=<API_KEY>
# Puis chaîner dans la liste des middlewares du routeur :
-
  traefik.http.routers.kesh.middlewares=kesh-crowdsec@docker,kesh-ratelimit@docker,kesh-header@docker
```

Installation locale CrowdSec : voir crowdsec.net. Inscription gratuite à la CTI communautaire pour partager les détections.

ModSecurity / OWASP Core Rule Set (CRS) — règles WAF traditionnelles Pour les organisations soumises à audit conformité (RGPD, LPD Suisse, exigences clients), un WAF basé sur OWASP CRS détecte les patterns d'attaque classiques (SQL injection, XSS, path traversal, etc.). Plusieurs intégrations sont disponibles :

- **Coraza** (plugin Traefik officiel WASM) — github.com/jcchavez/coraza-http-wasm. Engine WAF Go natif, performance élevée, règles CRS supportées.
- **ModSecurity v3 + Nginx** en frontal de Traefik — option plus complexe mais plus complète si déjà utilisé en interne.

4.4.4 Considérations sécurité spécifiques

- **Sock Docker exposé** (`/var/run/docker.sock`) : Traefik y accède en lecture seule (`:ro`). En haute exigence sécurité, préférer le `docker-proxy` (image [tecnativa/docker-socket-proxy](https://github.com/tecnativa/docker-socket-proxy)) qui filtre les endpoints Docker API accessibles à Traefik.
- **Renouvellement Let's Encrypt** : Traefik gère automatiquement le renouvellement 30 jours avant expiration. Surveiller via `docker logs kesh-traefik | grep acme`.

- **Backup de `acme.json`** : ce fichier contient la clé privée TLS — l'inclure dans votre stratégie de backup (volume `traefik-letsencrypt` ci-dessus).
- **Dashboard Traefik désactivé par défaut** dans cette config (pas de label `api` , pas d'option `--api`). Si vous l'activez en debug, protégez-le impérativement avec une auth + IP whitelist.

4.5 Alternative : déploiement sur Synology DSM

Synology DSM (DiskStation Manager) 7.2+ inclut le paquet *Container Manager* (anciennement *Docker*) qui permet de déployer Kesh sans utiliser le CLI Docker. Cette section couvre le scénario standard d'un déploiement Kesh sur un NAS Synology personnel ou de PME (DS220+, DS920+, DS1522+, DS1821+, ou modèles supérieurs avec ≥ 2 GiB RAM).

4.5.1 Prérequis Synology DSM

- **DSM** ≥ 7.2 (Container Manager n'est pas disponible sur DSM 6.x).
- **Modèle NAS** compatible Docker — vérifier sur la liste officielle Synology : modèles avec processeur Intel/AMD x86_64 (DS220+, DS720+, DS920+, DS1522+, DS1821+, etc.). Les modèles ARM (DS120j, DS220j, DS418, etc.) ne supportent pas Container Manager.
- **RAM** : minimum 2 GiB (4 GiB recommandés pour confort utilisateur multi-tenant avec plusieurs companies actives).
- **Stockage** : au moins 10 GiB libres sur le volume `/volume1` (5 GiB pour l'image Kesh + 1 GiB pour MariaDB initial + 4 GiB marge pour les imports bancaires, exports PDF/ZIP, et croissance données).
- **Paquet Container Manager** installé via Package Center (gratuit, fourni par Synology). Pour DSM 7.2+, c'est l'unique paquet officiel — il remplace l'ancien paquet *Docker*.

4.5.2 Étape 1 — Préparer la structure de répertoires

Via SSH (recommandé) ou File Station (alternative GUI), créer la structure suivante sous `/volume1/docker/kesh/` :

```
# Connexion SSH au NAS (activer SSH dans Panneau de configuration > Terminal
  et SNMP)
ssh admin@<IP-NAS>

# Création des dossiers
sudo mkdir -p /volume1/docker/kesh
cd /volume1/docker/kesh

# Permissions : utilisateur DSM courant + group docker (uid/gid varient
  selon DSM)
# Container Manager rendra les volumes accessibles automatiquement
sudo chown -R $(id -u):$(id -g) /volume1/docker/kesh
```

4.5.3 Étape 2 — Récupérer docker-compose.yml et .env

Télécharger le `docker-compose.yml` canonique depuis le dépôt Kesh et créer le fichier `.env` à partir du template (voir section 5.1 pour la liste complète) :

```
cd /volume1/docker/kesh
curl -fsSL
  https://raw.githubusercontent.com/guycorbaz/kesh/main/docker-compose.prod.yml
  -o docker-compose.yml
curl -fsSL
  https://raw.githubusercontent.com/guycorbaz/kesh/main/.env.example -o .env

# Générer les secrets obligatoires (DOIVENT remplacer les placeholders dans
# .env)
openssl rand -hex 32      # → KESH_JWT_SECRET
openssl rand -base64 24   # → KESH_ADMIN_PASSWORD
openssl rand -base64 32   # → MARIADB_ROOT_PASSWORD et MARIADB_PASSWORD

# Éditer .env avec vi/nano pour remplacer les <GENERATE_ME: ...> et <EDIT:
# ...>
sudo nano .env
```

! Attention

Sécurité : ne jamais laisser les valeurs par défaut `changeme` ou `change-me-32-bytes-...` dans `.env`. Story 10-1 du hardening Kesh fait fail-fast au boot si ces placeholders restent — c'est intentionnel pour protéger les utilisateurs débutants qui oublient cette étape.

4.5.4 Étape 3 — Importer le projet dans Container Manager

Méthode GUI (recommandée pour débutants)

1. Ouvrir **Container Manager** depuis le menu DSM.
2. Aller dans l'onglet **Projet** (panneau gauche).
3. Cliquer **Créer**.
4. Renseigner :
 - **Nom du projet** : `kesh`
 - **Chemin** : naviguer vers `/volume1/docker/kesh`
 - **Source** : *Utiliser docker-compose.yml existant*
5. Container Manager affiche un preview du compose. Vérifier que les services attendus apparaissent (`kesh-api` , `mariadb` , éventuellement `traefik` si vous l'avez ajouté).
6. Cliquer **Suivant** → **Démarrer le projet maintenant** → **Terminé**.

```
cd /volume1/docker/kesh

# Container Manager DSM expose `docker compose` (plugin v2)
sudo docker compose pull
sudo docker compose up -d

# Vérifier l'état
sudo docker compose ps
sudo docker compose logs -f kesh-api # Ctrl+C pour quitter
```

4.5.5 Étape 4 — Configurer le reverse proxy DSM (Application Portal)

Méthode CLI (équivalente, scriptable) DSM intègre nativement un reverse proxy HTTPS via le *Portail des applications*. Cette option est plus simple que Nginx/Caddy/Traefik pour les petites installations qui n'exposent pas Kesh à Internet (LAN-only).

1. Ouvrir **Panneau de configuration** → **Portail de connexion** → onglet **Avancé** → bouton **Proxy inversé**.
2. Cliquer **Créer**.
3. Renseigner :
 - **Nom de la description** : `Kesh comptabilité`
 - **Source** : Protocole `HTTPS` , Hostname `kesh.local` (ou votre domaine), Port `443`
 - **Destination** : Protocole `HTTP` , Hostname `localhost` , Port `80`
4. Onglet **En-têtes personnalisés** → ajouter :
 - `X-Forwarded-Proto` : `https`
 - `X-Real-IP` : `$remote_addr`
5. Onglet **Avancé** → cocher `HTTP/2` et `HSTS` (Strict-Transport-Security).
6. Cliquer **Enregistrer**.

Certificat HTTPS via DSM Let's Encrypt DSM intègre un client ACME pour Let's Encrypt :

1. **Panneau de configuration** → **Portail de connexion** → onglet **Certificat**.
2. Cliquer **Ajouter** → **Obtenir un certificat de Let's Encrypt**.
3. Renseigner le domaine (`kesh.exemple.ch`), l'email, et un *Subject Alternative Name* optionnel.
4. DSM valide le challenge HTTP-01 ou DNS-01 selon configuration, et installe le certificat automatiquement.
5. Lier le certificat au hostname dans **Paramètres** → sélectionner le service `kesh.exemple.ch` et choisir le nouveau certificat.

Note

Pour les déploiements exposés sur Internet avec besoin de firewall applicatif (rate limiting, WAF), préférer **Traefik en sidecar** (section 4.4) au Portail DSM. Le Portail couvre les besoins LAN-only ou les déploiements simples derrière un VPN/WireGuard.

4.5.6 Étape 5 — Vérification post-installation

Tester l'instance via le hostname configuré :

```
# Healthcheck (doit retourner JSON avec db:true)
curl -k https://kesh.local/health

# Réponse attendue (Story 10-3) :
# {"status":"ok", "db":true, "version":"0.1.0"}
```

Ouvrir un navigateur sur <https://kesh.local> et se connecter avec le compte admin renseigné dans `.env`. Le wizard d'onboarding démarre automatiquement (création de la première company, choix du plan comptable, configuration des exercices).

4.5.7 Limitations Container Manager DSM

- **Auto-restart** : Container Manager redémarre automatiquement les containers au reboot DSM (équivalent `restart: unless-stopped` dans compose). Pas de configuration supplémentaire requise.
- **Logs** : accessibles via Container Manager GUI (onglet *Container* → sélectionner `kesh-api` → *Détails* → *Journaux*) ou via SSH `docker compose logs -f`.
- **Mises à jour Kesh** : nécessitent un `docker compose pull && docker compose up -d` via SSH (Container Manager GUI ne propose pas de rebuild automatique sur push d'image). Voir section 8 pour la procédure complète.
- **Backup et restore** : voir section dédiée 7.7 pour les méthodes natives DSM (Hyper Backup, Snapshot Replication).

5 Configuration

5.1 Variables d'environnement complètes

Toute la configuration Kesh passe par des variables d'environnement, conformément à la philosophie [Twelve-Factor App](#). Le fichier `.env` est chargé automatiquement par Docker Compose.

5.1.1 Variables obligatoires

Variable	Description
JWT_SECRET	Secret de signature des JWT. Doit faire au moins 64 caractères . Générer avec <code>openssl rand -hex 64</code> .
MARIADB_ROOT_PASSWORD	Mot de passe root MariaDB. Utilisé uniquement à l'initialisation.
MARIADB_PASSWORD	Mot de passe utilisateur applicatif (kesh).
MARIADB_DATABASE	Nom de la base de données (par défaut : <code>kesh</code>).
MARIADB_USER	Nom de l'utilisateur applicatif (par défaut : <code>kesh</code>).
DATABASE_URL	URL complète de connexion DB. Format : <code>mysql://USER:PASS@HOST:PORT/DB</code> .
KESH_PUBLIC_URL	URL publique de l'instance (HTTPS). Utilisée dans les emails, QR Bills, liens partagés.

5.1.2 Variables optionnelles avec valeurs par défaut

Variable	Description	Défaut
<code>KESH_HOST</code>	Adresse d'écoute de l'API.	<code>0.0.0.0</code>
<code>KESH_PORT</code>	Port d'écoute de l'API.	<code>80</code>
<code>KESH_LOG_LEVEL</code>	Niveau de log (trace, debug, info, warn, error).	<code>info</code>
<code>KESH_LOG_FORMAT</code>	Format des logs (json, pretty).	<code>json</code>
<code>JWT_EXPIRY_SECONDS</code>	Durée de vie d'un access token.	<code>900 (15 min)</code>
<code>JWT_REFRESH_EXPIRY_DAYS</code>	Durée de vie d'un refresh token.	<code>30</code>
<code>KESH_CORS_ORIGINS</code>	Origines CORS autorisées (séparées par virgule).	<code>KESH_PUBLIC_URL</code>
<code>KESH_RATE_LIMIT_PER_MINUTE</code>	Nombre de requêtes max par IP par minute.	<code>300</code>
<code>KESH_SESSION_COOKIE_SECURE</code>	Force le flag <code>Secure</code> sur les cookies.	<code>true</code>
<code>KESH_DEFAULT_LOCALE</code>	Langue par défaut de l'instance (fr, de, it, en).	<code>fr</code>
<code>KESH_DEFAULT_TIMEZONE</code>	Timezone par défaut.	<code>Europe/Zurich</code>
<code>KESH_DEFAULT_CURRENCY</code>	Devise par défaut.	<code>CHF</code>
<code>KESH_MAX_UPLOAD_BYTES</code>	Taille max upload (pièces, CAMT.053).	<code>52428800 (50 Mo)</code>
<code>KESH_FEATURE_FLAGS</code>	Liste de feature flags activés.	<code>vide</code>

5.1.3 Variables liées au monitoring (optionnelles)

Variable	Description
<code>KESH_METRICS_ENABLED</code>	Active l'endpoint Prometheus <code>/metrics</code> .
<code>KESH_METRICS_PORT</code>	Port d'écoute pour les métriques (séparé).
<code>KESH_SENTRY_DSN</code>	DSN Sentry pour le tracking d'erreurs (optionnel, externe).

5.2 Configuration des logs fichier

Depuis la version 0.1.1, Kesh peut écrire ses logs dans un fichier avec rotation native, **en plus** de la sortie standard consultable via `docker logs kesh-api`. C'est utile car la rétention de `docker logs` est purgée à chaque recréation du conteneur, alors qu'un fichier persiste et peut être inclus dans le backup.

En production (`docker-compose.prod.yml`), cette sortie est **activée par défaut** : les logs sont écrits dans `/var/log/kesh/kesh.log` à l'intérieur du conteneur, monté sur le répertoire `./log/` de l'hôte (à côté du `.env` et de la base, donc couvert par le même backup).

Variable	Description	Défaut
<code>KESH_LOG_FILE_PATH</code>	Chemin complet du fichier de log. Vide = logs fichier désactivés (sortie standard uniquement).	<code>/var/log/kesh/kesh.log</code> (en prod)
<code>KESH_LOG_FILE_ROTATION</code>	Stratégie de rotation : <code>daily</code> , <code>hourly</code> ou <code>never</code> .	<code>daily</code>
<code>KESH_LOG_FILE_MAX_FILES</code>	Nombre de fichiers conservés. Ignoré si rotation = <code>never</code> .	<code>7</code>
<code>KESH_LOG_FILE_FORMAT</code>	Format : <code>pretty</code> (lisible) ou <code>json</code> (ingestion outillée).	<code>pretty</code>

! Attention

Le conteneur `kesh-api` s'exécute en tant que `root`, donc le répertoire `./log/` et ses fichiers appartiennent à `root:root` sur l'hôte NAS. Hyper Backup (qui tourne en `root`) les sauvegarde sans problème. Pour consulter les logs avec un compte non-`root`, utilisez `sudo tail -f ./log/kesh.log` ou ajustez les permissions du répertoire.

Pour **désactiver** les logs fichier (sortie standard uniquement), définir `KESH_LOG_FILE_PATH=` (valeur vide) dans le `.env`.

5.3 Configuration MariaDB

5.3.1 Paramètres recommandés

Modifier la configuration MariaDB pour optimiser l'usage Kesh. Créer `/etc/mysql/mariadb.conf.d/99-kesh.cnf`.

```
[mariadb]
# Encodage strict pour la conformité Unicode (factures multilingues)
character-set-server = utf8mb4
collation-server     = utf8mb4_unicode_ci

# Niveau d'isolation par défaut (REPEATABLE READ par défaut MariaDB,
# Kesh utilise des transactions explicites SERIALIZABLE pour les
# opérations critiques comme la numérotation des factures et la
# réconciliation bancaire)
transaction-isolation = REPEATABLE-READ

# Mode SQL strict pour éviter les coercions silencieuses
sql_mode =
    STRICT_TRANS_TABLES,NO_ENGINE_SUBSTITUTION,ERROR_FOR_DIVISION_BY_ZERO

# InnoDB buffer pool (~ 50% de la RAM disponible)
innodb_buffer_pool_size = 1G
innodb_log_file_size    = 256M
innodb_flush_log_at_trx_commit = 1
innodb_flush_method      = O_DIRECT

# Logs lents pour audit performance
slow_query_log           = 1
slow_query_log_file      = /var/log/mysql/slow.log
long_query_time           = 2

# Binlog pour replication ou point-in-time recovery
log_bin                  = /var/log/mysql/mariadb-bin
binlog_format             = ROW
expire_logs_days         = 14
```

! Astuce

Pour une PME standard (1 société, < 50'000 écritures/an), les paramètres par défaut MariaDB suffisent largement. Cette configuration avancée n'est utile qu'à partir de plusieurs sociétés ou de très gros volumes.

5.3.2 Initialisation manuelle de la base

Si vous ne passez pas par Docker Compose, voici comment initialiser manuellement la DB :

```
mariadb -u root -p <<SQL
CREATE DATABASE kesh CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci;
CREATE USER 'kesh'@'%' IDENTIFIED BY 'mot_de_passe_fort';
```



```
GRANT ALL PRIVILEGES ON kesh.* TO 'kesh'@'%';  
FLUSH PRIVILEGES;  
SQL
```

Au premier démarrage, l'application exécute les migrations Diesel.

5.4 Configuration multi-tenant

Kesh supporte plusieurs sociétés (*companies*) sur une même instance, avec isolation stricte des données.

5.4.1 Concept de *company*

Une *company* représente une entité comptable (PME, association, fondation). Chaque company possède :

- Un nom commercial et une raison sociale.
- Un numéro IDE (Identifiant Unique des Entreprises suisse).
- Une langue et timezone par défaut.
- Un plan comptable propre.
- Un ou plusieurs utilisateurs (avec rôles).
- Un ou plusieurs exercices comptables.
- Ses propres factures, contacts, écritures, transactions bancaires, rapports.

5.4.2 Isolation des données

Toutes les requêtes API et toutes les requêtes SQL Kesh sont filtrées par `company_id` (cf. Story Epic 7-1 «audit multi-tenant scoping refactor»). Un utilisateur de la company A ne peut en aucun cas accéder aux données de la company B, même via une URL forgée (défense en profondeur contre IDOR).

5.4.3 Cas d'usage fiduciaire

Un fiduciaire peut créer une seule instance Kesh avec N companies, une par dossier client. L'isolation par `company_id` garantit la confidentialité inter-client. Le fiduciaire peut être membre de toutes les companies avec un rôle d'administrateur ; chaque utilisateur PME ne voit que sa propre company.

i Note

La création d'une company se fait via l'interface utilisateur après connexion d'un administrateur. Voir le *Manuel utilisateur* pour les détails opérationnels.

6 Premier démarrage

6.1 Création du compte administrateur initial (onboarding self-service)

Depuis la version **v0.1.2**, Kesh adopte l'approche *self-service* commune aux applications self-hosted modernes (Jellyfin, Bitwarden, Vaultwarden) : au tout premier démarrage sur une base de données vide, un écran web `/setup` accueille l'administrateur pour créer son compte via un formulaire — sans édition préalable du fichier `.env`.

Le *bootstrap* crée seulement une **company provisoire** (placeholder) au boot. L'administrateur est ensuite créé via le formulaire `POST /api/v1/setup/admin` (route publique gated par `user_count == 0` → désactivée automatiquement en 410 Gone dès le 1^{er} admin créé).

6.1.1 Procédure (nouveau flow recommandé v0.1.2+)

1. Démarrer la stack (`docker compose up -d`) **sans** renseigner `KESH_ADMIN_*` dans `.env`.
2. Ouvrir `https://compta.exemple.ch` dans un navigateur. L'écran « **Bienvenue dans Kesh** » apparaît automatiquement.
3. Saisir un nom d'utilisateur (ex. `admin`), un mot de passe fort (≥ 12 caractères) et sa confirmation.
4. Cliquer sur « **Créer le compte administrateur** ».
5. Le wizard d'onboarding démarre automatiquement (langue, mode, coordonnées de la société, banque).

! Attention

Sécurité — bloquer l'accès réseau public avant le 1^{er} démarrage. La personne qui touche `/api/v1/setup/admin` en premier devient administrateur. Recommandé : binder loopback `127.0.0.1` (mapping Docker par défaut en prod) ou LAN privé via reverse proxy en attendant la création du compte. Le rate-limit IP (5 tentatives/15 min) limite le brute-force du formulaire.

6.1.2 Procédure alternative — bootstrap déclaratif via `.env`

Pour les déploiements automatisés (CI, Test, scripts de provisioning), il reste possible de créer l'administrateur *déclarativement* via les variables `.env` — équivalent strict du comportement v0.1.0/v0.1.1 :

1. Renseigner `KESH_ADMIN_USERNAME=admin` et `KESH_ADMIN_PASSWORD=<mot-de-passe-fort-12-chars>` dans `.env` *avant* le premier démarrage.
2. Démarrer la stack (`docker compose up -d`). Le boot crée la company stub **et** l'administrateur

en une seule passe.

3. Se connecter directement avec ces identifiants — pas d'écran `/setup`.
4. **Retirer les variables** `KESH_ADMIN_*` de `.env` après la première connexion réussie (sinon, chaque redémarrage logge un warning « retirer les vars »).
5. Changer le mot de passe administrateur via l'UI dès que possible.

! Astuce

Le flow self-service `/setup` est préférable pour les déploiements interactifs — il évite la mauvaise pratique consistant à laisser un mot de passe en clair dans `.env`.

6.2 J'ai oublié mon mot de passe administrateur (recovery break-glass)

Depuis la version **v0.1.2**, Kesh permet de réinitialiser le mot de passe administrateur sans accès à la base de données. Le mécanisme *recovery break-glass* réutilise les variables `KESH_ADMIN_*` dans un double-usage : sur une instance déjà configurée (au moins un user existe), renseigner ces variables avec un mot de passe différent du hash actuel déclenche un **reset du password de l'administrateur correspondant au username configuré**.

6.2.1 Procédure step-by-step

1. **Stopper le container** : `docker compose stop kesh-api` (la DB reste accessible pour les autres clients).
2. **Éditer** `.env` : décommenter (ou ajouter) les deux variables :

```
KESH_ADMIN_USERNAME=admin          # le username de votre compte admin
KESH_ADMIN_PASSWORD=<nouveau-mdp-12+> # nouveau mot de passe (>= 12 chars)
```

3. **Redémarrer le container** : `docker compose up -d kesh-api`. Au boot, les logs affichent :

```
WARN bootstrap: Recovery break-glass declenche pour user 'admin'...
ERROR bootstrap: Recovery effectue -- RETIRER LES VARS DE .ENV
```

4. **Se connecter** via l'écran de login avec le nouveau mot de passe. Les sessions actives sur ce compte ont été révoquées par le recovery — les autres onglets/devices devront se reconnecter.
5. **Retirer les variables** `KESH_ADMIN_*` de `.env` (les recommenter ou les supprimer). Tant qu'elles restent en place, un `WARN bootstrap: retirer les vars de .env` apparaît à chaque redémarrage (no-op silencieux côté DB grâce à la détection « hash identique »).
6. (Optionnel) Redémarrer une dernière fois (`docker compose restart kesh-api`) pour confirmer que les warnings bootstrap ont disparu des logs — preuve que les variables `KESH_ADMIN_*` sont bien retirées.

! Attention

Sécurité du fichier `.env`. Tant que `KESH_ADMIN_PASSWORD` est non-vide dans `.env`, toute personne ayant accès à ce fichier peut reset le mot de passe de l'administrateur. Restreindre les permissions : `chmod 600 .env`. Le recovery génère également une entrée `admin_break_glass_reset` dans le journal d'audit (`audit_log`, conservation 10 ans Swiss CO Art. 958f) pour traçabilité.

6.2.2 Matrice de comportement au boot — vue d'ensemble

Selon l'état de la base de données et la présence des variables `KESH_ADMIN_*`, le bootstrap se comporte ainsi :

users	env set ?	match ?	Comportement
vide	non	—	Crée la company stub seule, écran <code>/setup</code> actif
vide	oui	—	Crée stub + admin déclaratif (équivalent v011-2)
existant	non	—	No-op (régime nominal post-bootstrap)
existant	oui	match + hash identique	No-op silencieux, warning « retirer les vars »
existant	oui	match + hash différent	RECOVERY : reset password + révocation tokens + <code>audit_log</code>
existant	oui	no match username	No-op + warning « no user matches »

6.3 Déploiement LAN HTTP-only (sans HTTPS) — v0.1.3+

Contexte : avant v0.1.3, Kesh émettait les cookies de session avec le flag `Secure` en mode production. Le navigateur refuse de stocker ces cookies sur une connexion HTTP non-TLS, rendant l'application inutilisable sur un déploiement LAN strict sans HTTPS (par exemple un NAS Synology privé sur un domaine RFC 8375 `*.home.arpa` derrière un reverse proxy Traefik en HTTP).

Depuis la version **v0.1.3**, la variable d'environnement `KESH_COOKIE_SECURE` permet de désactiver explicitement le flag `Secure` pour ces cas d'usage.

6.3.1 Décision : HTTPS ou HTTP-only ?

Contexte

Recommandation

Prod Internet exposée

HTTPS obligatoire
(Let's Encrypt automatique via Traefik / Caddy / Synology DSM).
`KESH_COOKIE_SECURE` reste à la valeur défaut `true`.

NAS privé LAN avec domaine externe (DDNS, Cloudflare, OVH)

HTTPS recommandé
(Let's Encrypt via DNS challenge).
`KESH_COOKIE_SECURE=true`.

NAS privé LAN avec domaine RFC 8375 (`*.home.arpa`)

2 options : (a) certificat auto-signé avec `mkcert` + Traefik file provider (browser warning à accepter une fois par poste, cookies Secure marchent) — **recommandé**.
(b) HTTP-only avec `KESH_COOKIE_SECURE=false`, à n'utiliser que si le LAN est strictement de confiance.

Dev local pur (cargo run sans Docker)

`KESH_COOKIE_SECURE=false` pour itération rapide. Ne jamais déployer cette configuration en production.

6.3.2 Procédure HTTP-only (`KESH_COOKIE_SECURE=false`)

1. Éditer `.env` :

```
KESH_COOKIE_SECURE=false
```

2. Redémarrer le container : `docker compose restart kesh-api`.
3. Les logs au boot affichent un warning explicite rappelant le compromis sécurité :

```
WARN config: KESH_COOKIE_SECURE=false -- cookies session emis SANS le
flag `Secure`. Acceptable uniquement pour deploiement HTTP-only LAN
strict (e.g. domaine RFC 8375 `*.home.arpn`, NAS prive). NE JAMAIS
activer en prod exposee Internet sans HTTPS -- cookies sniffables
sur tout reseau non-TLS.
```

! Attention

Risque de sécurité avec `KESH_COOKIE_SECURE=false`. Les cookies de session sont émis sans le flag `Secure` et transitent donc en clair sur le réseau HTTP. Un attaquant ayant accès au LAN (`tcpdump`, point d'accès WiFi compromis, switch managé miroir) peut intercepter les cookies et usurper la session administrateur. À n'activer que si le LAN est strictement de confiance et physiquement isolé d'Internet. Ne jamais combiner avec une exposition Internet (port forwarding, reverse-proxy externe, etc.) sans HTTPS.

6.3.3 Valeurs acceptées et fail-fast

`KESH_COOKIE_SECURE` accepte strictement `true` / `1` (par défaut, cookies Secure activés) ou `false` / `0` (cookies sans Secure). Toute autre valeur (`True`, `TRUE`, `yes`, `on`, espaces avant/après, etc.) refuse le démarrage avec un message d'erreur explicite. Ce parsing strict (cohérent avec `KESH_TEST_MODE`) évite l'ambiguïté qui ferait croire qu'un override est actif alors qu'il ne l'est pas.

6.4 Configuration de la company via l'onboarding

Une fois connecté, l'**assistant d'onboarding** (wizard) configure la company provisoire — il n'y a pas d'écran séparé de création de société :

1. **Langue** de l'interface, puis **mode** (guidé ou expert).
2. **Parcours** : données de démonstration (exploration) ou configuration de production.
3. **Type d'organisation** (indépendant, association, PME) et **langue comptable**.
4. **Coordonnées** de l'organisation (raison sociale, adresse, numéro IDE) — ceci renomme la company provisoire et lève la bannière « nom provisoire ».
5. **Compte bancaire** principal (optionnel, configurable plus tard).

Tant que les coordonnées ne sont pas renseignées, une bannière non-bloquante rappelle que l'entreprise porte un nom provisoire (correctif catch-22 fresh-install, v0.1.1).

6.5 Choix du plan comptable

Kesh inclut plusieurs plans comptables suisses pré-configurés :

Plan	Description
Sterchi PME	Plan comptable standard PME suisse, version Sterchi (la plus répandue). Recommandé pour la majorité des cas.
KMU	Plan KMU (Käfer / Treuhand-Kammer), version simplifiée orientée artisans et indépendants.
Personnalisé CSV	Import d'un plan personnalisé depuis un fichier CSV (format spécifié dans le manuel utilisateur).

! Astuce

Le plan comptable peut être modifié après création (ajout de comptes auxiliaires, sous-comptes, etc.) mais sa structure globale reste stable. Choisir le plan le plus proche de votre activité dès la création évite des migrations ultérieures.

6.6 Configuration des exercices comptables

Chaque company doit définir au moins un exercice comptable (*fiscal year*). Par défaut, l'exercice civil 1^{er} janvier – 31 décembre est créé automatiquement à la création de la company. Pour d'autres exercices (décalés, courts, longs), aller dans *Paramètres* → *Exercices*.

6.7 Création des utilisateurs et rôles

Kesh utilise un système de contrôle d'accès basé sur les rôles (RBAC) :

Rôle	Permissions
super_admin	Accès total à toutes les companies, gestion des sociétés, gestion des utilisateurs cross-company. Réservé au fiduciaire ou à l'administrateur système.
admin	Gestion complète d'une company (utilisateurs, paramètres, plan comptable, exercices).
accountant	Saisie d'écritures, validation, modification, import bancaire, réconciliation, génération de factures et rapports.
operator	Saisie d'écritures en brouillon, génération de factures, consultation des rapports. Pas de validation finale ni de modification d'écritures validées.
viewer	Lecture seule. Consultation des rapports, factures, écritures, sans modification possible.

Création d'un utilisateur via l'UI : *Paramètres* → *Utilisateurs* → *Inviter*.

7 Sauvegarde et restauration

7.1 Importance de la sauvegarde

La sauvegarde régulière de l'instance Kesh est **critique** pour deux raisons :

- 1. Continuité d'activité** : perte de données = arrêt de la comptabilité PME.
- 2. Conformité OLICo Art. 6** (Disponibilité) : « jusqu'à la fin du délai de conservation [10 ans], toute personne autorisée doit pouvoir, en tout temps et dans un délai raisonnable, consulter et vérifier les livres et les pièces comptables ». Une perte de données rend l'instance non-conforme.

7.2 Stratégie recommandée

- **Quotidienne** : `mariadb-dump` complet, conservé 30 jours, stocké en local + offsite (cloud).
- **Hebdomadaire** : snapshot du volume Docker `/var/lib/docker/volumes/kesh_db_data` (gel cohérent via `docker compose stop` puis snapshot, ou via LVM/ZFS).
- **Mensuelle** : export ZIP de souveraineté (Story 9-2b) pour chaque company depuis l'interface utilisateur.
- **Annuelle** : archivage long terme conforme OLICo Art. 9 (support non-modifiable type CD-R,

ou hash signé sur support modifiable WORM).

7.3 Script de sauvegarde mariadb-dump

Voici un script bash de référence à exécuter via cron quotidien :

```
#!/bin/bash
# /usr/local/bin/kesh-backup.sh
# Sauvegarde quotidienne de la base Kesh

set -euo pipefail

BACKUP_DIR="/var/backups/kesh"
RETENTION_DAYS=30
TIMESTAMP=$(date +%Y%m%d_%H%M%S)
BACKUP_FILE="${BACKUP_DIR}/kesh_${TIMESTAMP}.sql.gz"

mkdir -p "${BACKUP_DIR}"

# Dump avec gzip et stockage hash SHA-256
docker compose exec -T db \
    mariadb-dump \
    --single-transaction \
    --routines \
    --triggers \
    --events \
    --add-drop-database \
    --databases kesh \
    -u root \
    -p"${MARIADB_ROOT_PASSWORD}" \
    | gzip > "${BACKUP_FILE}"

# Hash SHA-256 pour verification ulterieure (OLiCo Art. 9 al. 1.b ch. 1)
sha256sum "${BACKUP_FILE}" > "${BACKUP_FILE}.sha256"

# Conservation 30 jours
find "${BACKUP_DIR}" -name "kesh_*.sql.gz" -mtime +${RETENTION_DAYS} -delete
find "${BACKUP_DIR}" -name "kesh_*.sql.gz.sha256" -mtime +${RETENTION_DAYS} \
    -delete

echo " Sauvegarde terminee : ${BACKUP_FILE}"
```

Crontab d'exemple (sauvegarde à 2h du matin chaque jour) :

```
0 2 * * * /usr/local/bin/kesh-backup.sh >> /var/log/kesh-backup.log 2>&1
```

7.4 Stockage offsite

Synchroniser les sauvegardes vers un stockage distant (S3, Backblaze B2, Wasabi, Hetzner Storage Box, OVH C14, etc.) :

```
# Exemple avec rclone
rclone sync /var/backups/kesh remote:kesh-backups \
  --transfers 4 \
  --checksum \
  --log-file /var/log/kesh-backup-sync.log
```

7.5 Restauration depuis sauvegarde

```
# 1. Verifier l'integrite de la sauvegarde (SHA-256)
cd /var/backups/kesh
sha256sum -c kesh_20260520_020000.sql.gz.sha256

# 2. Arrêter Kesh (pas la DB)
docker compose stop kesh

# 3. Restaurer la base
zcat kesh_20260520_020000.sql.gz | docker compose exec -T db \
  mariadb -u root -p"${MARIADB_ROOT_PASSWORD}"

# 4. Redémarrer Kesh
docker compose start kesh

# 5. Verifier le fonctionnement
curl http://localhost/health
```

7.6 Test de restauration périodique (OLICo Art. 10 al. 1)

L'article 10 al. 1 OLICo exige que « l'intégrité et la lisibilité des supports d'information sont régulièrement contrôlées ». Cela implique de **tester effectivement la restauration** au moins **annuellement**, sur un environnement de test isolé :

1. Provisionner une instance Kesh de test (machine virtuelle ou container isolé).
2. Restaurer la sauvegarde la plus récente.
3. Vérifier que l'application démarre, que les écritures sont accessibles, qu'un export ZIP de souveraineté peut être généré et que son SHA-256 est identique à celui généré sur l'instance de production avant la sauvegarde.
4. Documenter le test dans un procès-verbal écrit (date, opérateur, résultat, anomalies éventuelles).

Conservation 10 ans avec les sauvegardes.

! Astuce

Automatisez ce test annuel via un workflow CI/CD : provisioning Docker éphémère, restauration, exécution de tests E2E Playwright basiques, génération du procès-verbal. Cette automatisation peut être traitée comme une story Epic 10 « Déploiement & Opérations » ou Epic ultérieur.

7.7 Backup natif sur Synology DSM (Hyper Backup + Snapshot Replication)

Pour les installations sur Synology DSM (cf. section 4.5), DSM intègre nativement deux outils de backup éprouvés : **Hyper Backup** (backup incrémental versionné, idéal cloud + LAN) et **Snapshot Replication** (snapshot Btrfs point-in-time, idéal recovery instantané). Ces outils remplacent ou complètent le script `mariadb-dump` CLI ci-dessus selon votre stratégie.

7.7.1 Hyper Backup — backup incrémental + offsite intégré

Hyper Backup est l'outil principal Synology pour les backups versionnés vers stockage local externe (HDD USB, NAS distant), cloud (Synology C2 Backup, S3, Backblaze B2, Wasabi, Google Drive, OneDrive, etc.), ou serveur rsync distant. Il supporte la déduplication, le chiffrement client-side AES-256, et la rotation Smart (équivalent grandfather-father-son).

Configuration Hyper Backup pour Kesh

1. Installer **Hyper Backup** via Package Center (gratuit).
2. Lancer Hyper Backup → + **Tâche de sauvegarde de données** → choisir la destination (ex. *Synology C2 Backup* pour offsite managé, *S3 Storage* pour AWS/Wasabi/Backblaze, *Dossier local* pour HDD USB attaché au NAS).
3. Cocher les dossiers à backup :
 - **Obligatoire** : `/volume1/docker/kesh/` (compose + `.env` + tous les volumes via bind-mount si vous utilisez le pattern Synology recommandé).
 - **Optionnel** : volumes Docker nommés si vous n'utilisez pas le bind-mount (rare sur DSM) — accessibles via `/volume1/@docker/volumes/kesh_*`.
4. Cocher l'application **Container Manager** dans *Applications* pour backup automatique de la configuration des projets.
5. Activer le **chiffrement client-side** (mot de passe différent du `KESH_JWT_SECRET`, stocké séparément offline — la perte de ce mot de passe rend les backups illisibles).
6. Configurer la rotation **Smart Recycle** : recommandé *Conservez toutes les versions des derniers 30 jours + 1 par semaine pour 12 semaines + 1 par mois pour 12 mois*. Convertit OLICo 10 ans en pratique via archivage annuel séparé (cf. *Stockage offsite* ci-dessus).

7. Planifier la fréquence : **quotidienne à 2h du matin** (équivalent du cron du script CLI).
8. Activer les notifications email/push DSM pour alerter d'un échec de backup.

Cohérence transactionnelle MariaDB Hyper Backup copie les fichiers data au niveau système sans coordination avec MariaDB. Pour garantir un dump cohérent (pas de transactions partielles capturées), utiliser un **pre-script Hyper Backup** qui dump MariaDB juste avant le backup, et un post-script qui nettoie après :

```
#!/bin/bash
# /volume1/docker/kesh/pre-backup.sh
# Configure dans Hyper Backup > Parametres > Pre-script

set -euo pipefail
cd /volume1/docker/kesh

# Dump MariaDB juste avant Hyper Backup capture les fichiers
sudo docker compose exec -T mariadb \
    mariadb-dump --single-transaction --routines --triggers --events \
    --add-drop-database --databases kesh \
    -u root -p"${(grep MARIADB_ROOT_PASSWORD .env | cut -d= -f2)}" \
    | gzip > /volume1/docker/kesh/kesh_pre_backup.sql.gz

sha256sum /volume1/docker/kesh/kesh_pre_backup.sql.gz \
    > /volume1/docker/kesh/kesh_pre_backup.sql.gz.sha256
```

Le post-script peut être laissé vide ou utilisé pour supprimer le dump intermédiaire si on préfère minimiser l'espace disque local (au prix de devoir restorer via Hyper Backup pour recovery).

i Note

Pour les charges légères (mono-fiduciaire, < 100 écritures/jour), le simple backup des volumes Docker via Hyper Backup sans pre-script suffit dans 99% des cas — MariaDB redémarre depuis un état semi-cohérent à la restauration, et les éventuelles transactions perdues correspondent à la fraction de seconde du backup. Le pre-script devient utile pour les fiduciaires avec saisie 9-17h continue.

7.7.2 Snapshot Replication — recovery instantané point-in-time

Snapshot Replication fournit des snapshots Btrfs natifs du volume `/volume1/` avec recovery quasi-instantané (< 1 minute) et point-in-time precision (jusqu'à 1 snapshot par 5 minutes). Idéal pour se prémunir contre erreurs humaines récentes (suppression accidentelle d'une écriture, corruption logique) où Hyper Backup serait trop coûteux à restorer (téléchargement cloud + déchiffrement + restore).

Prérequis Snapshot Replication

- Modèle NAS supportant **Btrfs** (DS220+, DS720+, DS920+, DS1522+, DS1821+ — vérifier la liste officielle Synology).
- Volume principal formaté en **Btrfs** (pas EXT4). Si déjà en EXT4, migration nécessaire (sauvegarde complète + recréation du volume + restore — opération longue, planifiée hors période d'activité).
- Paquet **Snapshot Replication** installé via Package Center.

Configuration Snapshot Replication pour Kesh

1. Ouvrir **Snapshot Replication** → onglet **Snapshots**.
2. Sélectionner le dossier partagé `docker` (qui contient `/volume1/docker/kesh`).
3. **Paramètres** → onglet **Snapshot** → activer *Programmer la création de snapshots automatiques*.
4. Fréquence recommandée : **toutes les heures, du lundi au vendredi 6h-22h** (couvre les périodes de saisie active fiduciaire).
5. Rotation : conserver **72 snapshots horaires** (3 jours d'historique) + **14 snapshots quotidiens** + **12 snapshots hebdomadaires**.
6. Activer *Visible aux utilisateurs* si vous voulez permettre aux utilisateurs DSM de naviguer les snapshots via File Station (utile pour récupérer manuellement un fichier supprimé sans intervention admin).

Recovery depuis Snapshot

1. Snapshot Replication → **Snapshots** → sélectionner le dossier `docker`.
2. Choisir le snapshot avec le timestamp pre-incident.
3. **Actions** → **Restaurer** → *Restaurer dans un nouveau dossier* (recommandé, préserve l'état actuel pour comparaison) OU *Écraser le dossier original* (si certain).
4. Après restore : `cd /volume1/docker/kesh && sudo docker compose down && sudo docker compose up -` pour recharger l'instance Kesh sur le state restauré.

7.7.3 Stratégie 3-2-1 sur DSM

Combiner les outils pour atteindre la règle **3-2-1** (3 copies des données, sur 2 supports différents, dont 1 offsite) :

- **Copie 1 (primaire)** : volume Btrfs DSM avec Snapshot Replication actif (recovery rapide erreurs récentes).
- **Copie 2 (locale, support différent)** : Hyper Backup vers HDD USB externe attaché au NAS, rotation Smart Recycle 30 jours + 12 mois.
- **Copie 3 (offsite)** : Hyper Backup vers cloud (Synology C2 Backup pour le confort, ou S3/Backblaze B2 pour le coût optimal), chiffré client-side.
- **Archivage long terme (OLICo 10 ans)** : 1 dump annuel par year exporté manuellement sur média WORM (CD-R ou DVD-R inscriptible une fois) ou archivage tape LTO si volume.

! Attention

Test de restauration annuel obligatoire (cf. sous-section précédente OLICo Art. 10 al. 1) : applicable aux backups DSM aussi. Inclure dans le procès-verbal annuel la mention explicite de la méthode utilisée (Hyper Backup, Snapshot Replication) et le temps de recovery mesuré (RTO).

8 Mise à jour de Kesh

8.1 Process de release Kesh

Les versions de Kesh sont publiées sur GitHub avec un tag `vX.Y.Z` (versionnage sémantique). À chaque release :

- Une nouvelle image Docker `gcorbaz/kesh:X.Y.Z` et `gcorbaz/kesh:latest` sont poussées sur Docker Hub.
- Un changelog (`CHANGELOG.md`) liste les nouveautés, corrections de bugs et changements de compatibilité.
- Les manuels (admin, utilisateur, brochure marketing) sont rebuild en PDF et attachés au release GitHub.
- Les notes de breaking changes sont mises en évidence avec instructions de migration.

8.2 Suivre les notifications de release

- Activer les notifications GitHub : *Watch* le dépôt `guycorbaz/kesh` → *Custom* → *Releases*.
- Souscrire au flux RSS Atom des releases : <https://github.com/guycorbaz/kesh/releases.atom>.

8.3 Procédure de mise à jour standard

1. **Sauvegarder** la base de données et le volume `kesh_db_data` (cf. section précédente).
2. **Lire le changelog** de la nouvelle version pour identifier les breaking changes éventuels et les migrations nécessaires.
3. **Mettre à jour** `docker-compose.yml` si nécessaire (nouvelles env vars, changement d'image).
4. **Tirer la nouvelle image** : `docker compose pull kesh`.
5. **Redémarrer** : `docker compose up -d kesh`.
6. **Vérifier** les logs : `docker compose logs -f kesh` pour s'assurer que les migrations DB se sont bien déroulées.

7. **Tester** l'accès UI et les fonctionnalités critiques (login, saisie d'une écriture, génération d'une facture).

! Attention

Toujours sauvegarder **avant** la mise à jour. Une migration DB peut être irréversible. En cas de problème, la restauration depuis sauvegarde reste votre filet de sécurité.

8.4 Mise à jour majeure (changement de version X)

Une mise à jour majeure (par exemple v0.1 → v1.0) peut comporter des breaking changes. Procédure recommandée :

1. Lire intégralement le changelog et les notes de migration.
2. Provisionner une instance de test en parallèle de la production.
3. Restaurer une copie récente de la base de production sur l'instance de test.
4. Faire la mise à jour sur l'instance de test.
5. Valider toutes les fonctionnalités critiques sur l'instance de test.
6. Sauvegarder la production, puis appliquer la mise à jour.
7. Surveiller les logs pendant 24–48h post-update.

8.5 Rollback en cas d'échec

Si une mise à jour échoue ou révèle un bug bloquant :

```
# 1. Arrêter le service
docker compose stop kesh

# 2. Restaurer la version précédente de l'image
docker compose down
# Editer docker-compose.yml : image: gcorbaz/kesh:0.1.0 (ancien tag)
docker compose up -d

# 3. Si les migrations DB ont été appliquées et sont irréversibles, restaurer
# la sauvegarde DB d'avant la mise à jour (cf. section Restauration).
```

9 Sécurité

9.1 HTTPS obligatoire en production

Kesh **doit** être accessible uniquement via HTTPS en production. Le mode HTTP plain doit être réservé strictement au développement local.

Configurations recommandées du reverse proxy :

- TLS 1.2 minimum, TLS 1.3 préféré.
- Suites de chiffrement modernes uniquement (cf. [Mozilla SSL Config Generator](#)).
- HSTS activé avec `max-age=31536000; includeSubDomains`.
- Certificats Let's Encrypt (renouvellement automatique via Certbot ou Caddy intégré).
- Headers de sécurité : `X-Content-Type-Options: nosniff, X-Frame-Options: DENY, Referrer-Policy: strict-origin-when-cross-origin`.

9.2 Gestion des secrets

- **Jamais committer** `.env` dans Git. Ajouter à `.gitignore`.
- Utiliser un secrets manager (HashiCorp Vault, AWS Secrets Manager, Doppler, ou simplement `systemd-creds` sous Linux) pour les déploiements multi-instances.
- Permissions strictes sur `.env` : `chmod 600 .env` (lecture/écriture propriétaire uniquement).
- Rotation périodique du `JWT_SECRET` : tous les 6–12 mois en production. Une rotation invalide tous les tokens existants, forçant les utilisateurs à se reconnecter.

9.3 Authentification JWT + refresh tokens

Kesh utilise un modèle JWT avec rotation de refresh tokens :

- L'access token (JWT court, 15 min par défaut) est envoyé en cookie `HttpOnly; Secure; SameSite=Lax`.
- Le refresh token (durée 30 jours par défaut) est stocké en DB (table `refresh_tokens`) avec hash SHA-256 (jamais stocké en clair).
- À chaque refresh, le token précédent est invalidé (rotation), une trace est conservée pour détection d'attaque par replay.
- La déconnexion révoque tous les refresh tokens de la session.

9.4 Clés API (PAT) — accès programmatique

En complément de l'authentification web par cookie, Kesh permet à des **intégrations externes** (IA, scripts, ETL, dashboards BI, ERP) de consommer l'API via des **clés d'accès personnelles** (PAT — *Personal Access Token*). Une clé s'utilise sur les routes `/api/v1/*` existantes, présentée dans l'en-tête `Authorization: Bearer kesh_pat_...`

- **Gestion** : les clés se créent et se révoquent depuis l'interface web, page **Paramètres → Clés**

API (`/settings/api-keys`), accessible aux rôles Comptable et Administrateur. La gestion des clés est **impossible via l'API elle-même** (protection anti-auto-propagation : une clé compromise ne peut pas en créer d'autres).

- **Portée** : chaque clé est en *lecture seule* (`read` — méthodes GET uniquement) ou *lecture-écriture* (`read-write`). La permission effective est l'intersection de la portée de la clé et du rôle de son créateur.
- **Périmètre** : une clé est liée à **une seule entreprise** (*company*) et agit au nom de son créateur (rôle relu en base à chaque requête — désactiver le créateur invalide ses clés immédiatement).
- **Stockage** : seul un condensé SHA-256 de la clé est conservé en base (table `api_keys`) ; le secret en clair n'est affiché qu'une seule fois, à la création.
- **Expiration / révocation** : une clé peut avoir une date d'expiration optionnelle ; la révocation prend effet immédiatement.
- **Audit** : les mutations effectuées via une clé sont tracées dans `audit_log` avec `actor_type = 'api_key'` (et l'identifiant de la clé), conformément à l'OLICo Art. 9.

! Attention

Moindre privilège. Une clé `read-write` créée par un **Administrateur** hérite des pouvoirs d'Administrateur (gestion d'utilisateurs, réinitialisation de mots de passe, paramètres de facturation) — limitation connue de la v0.2 (suivi GitHub [KF-036](#) / [#167](#)). Ne créez de clés qu'avec le rôle minimal nécessaire, et privilégiez la portée `read` lorsque la lecture suffit. La v0.2 n'applique **pas** de limitation de débit (*rate-limiting*) par clé : utilisez l'expiration et la révocation pour limiter l'exposition.

Le guide d'intégration complet (exemples `curl`, Python, JavaScript, MCP, table des codes d'erreur) est fourni dans le dépôt à `docs/api-external.md`.

9.5 RBAC et permissions

Le contrôle d'accès basé sur les rôles est appliqué **au niveau de chaque endpoint API** via un middleware Tower (Axum). Les 5 rôles standards (cf. section Premier démarrage) sont définis dans le code (pas de rôles custom dynamiques v0.1).

9.6 Audit-trail (`audit_log`)

Toute action métier mutante est enregistrée dans la table `audit_log` :

- Contrainte **insert-only** : aucune route API ne permet de modifier ou supprimer une entrée `audit_log` (défense en profondeur).
- **Champs** : `user_id`, `company_id`, `timestamp`, `action`, `entity_type`, `entity_id`, `metadata_json`.
- Conservation 10 ans (selon CO Art. 958f al. 1).

9.7 Conformité OLICo Art. 9 — intégrité des supports modifiables

Conformément à l'analyse réglementaire (cf. [_bmad-output/planning-artifacts/research-swiss-co-958f.m](#)), Kesh satisfait l'OLICo Art. 9 al. 1.b par le couple :

- `audit_log` **immutable** insert-only (procédé technique garantissant l'intégrité, condition 1).
- SHA-256 dans `metadata.json` du ZIP d'export de souveraineté (Story 9-2b, condition 1 renforcée).
- Timestamp DB `created_at` pour preuve de moment d'enregistrement (condition 2).
- Documentation des procédures dans ce manuel + journal de bord (conditions 3 et 4).

i Note

La signature électronique qualifiée LSCSE (QES) n'est **pas requise** par l'OLICo pour les exports comptables des PME (interprétation « p. ex. » du législateur). Le verdict (b) « Dette explicite v0.2 » du document de recherche 9-5-4 confirme la conformité v0.1. Une implémentation QES est prévue en v0.2 via Epic 14 (issue GitHub #98).

9.8 Sécurité réseau (firewall)

Configuration firewall minimale (UFW sur Ubuntu/Debian) :

```
sudo ufw default deny incoming
sudo ufw default allow outgoing
sudo ufw allow 22/tcp comment 'SSH'
sudo ufw allow 80/tcp comment 'HTTP (redirect to HTTPS)'
sudo ufw allow 443/tcp comment 'HTTPS'
sudo ufw enable
```

9.9 Mises à jour de sécurité du système hôte

- Appliquer les mises à jour OS automatiquement (`unattended-upgrades` sur Debian/Ubuntu).
- Surveiller les CVE Docker (mise à jour de la version Docker Engine régulière).
- Surveiller les CVE MariaDB (mise à jour de l'image MariaDB lors des releases mineures).
- Souscrire aux annonces de sécurité Kesh (label `security` sur les issues GitHub).

10 Monitoring et logs

10.1 Logs applicatifs

Par défaut, Kesh écrit ses logs sur `stdout` en format JSON (variable `KESH_LOG_FORMAT=json`). Docker les capture et les rend accessibles via :

```
docker compose logs -f kesh
docker compose logs --tail=100 kesh
docker compose logs --since=1h kesh
```

10.2 Centralisation des logs

En production multi-instances, centraliser via :

- **ELK Stack** : Elasticsearch + Logstash + Kibana.
- **Loki + Grafana** : plus léger, recommandé pour les déploiements modestes.
- **Syslog distant** : configuration via le driver Docker logging `syslog`.

Exemple de driver logging Loki dans `docker-compose.yml` :

```
services:
  kesh:
    image: gcorbaz/kesh:latest
    logging:
      driver: loki
      options:
        loki-url: "https://loki.exemple.ch/loki/api/v1/push"
        loki-retries: "5"
        loki-batch-size: "400"
```

10.3 Métriques Prometheus

Activer l'endpoint metrics :

```
KESH_METRICS_ENABLED=true
KESH_METRICS_PORT=9090
```

Scraper avec Prometheus :

```
# prometheus.yml
scrape_configs:
  - job_name: 'kesh'
    static_configs:
```

```
- targets: ['localhost:9090']
```

Métriques exposées :

- `kesh_http_requests_total` : compteur requêtes HTTP par route + status.
- `kesh_http_request_duration_seconds` : histogramme durée des requêtes.
- `kesh_db_queries_total` : compteur requêtes DB.
- `kesh_active_users` : jauge utilisateurs actifs (sessions valides).
- `kesh_invoices_generated_total` : compteur factures générées.
- `kesh_imports_processed_total` : compteur imports CAMT.053 traités.

10.4 Alerting recommandé

Alertes Prometheus / AlertManager suggérées :

- Taux d'erreurs HTTP 5xx > 1% sur 5 minutes.
- Latence p95 > 2 secondes sur les requêtes API.
- Connexion DB perdue (échec `kesh_db_queries_total`).
- Disque DB > 80% utilisé.
- Backup quotidien échoué (exit code non-zéro).

11 Conformité légale suisse

11.1 Code des obligations Art. 957a — Tenue de la comptabilité

Kesh satisfait intégralement les exigences de l'Art. 957a CO pour les PME suisses. Les détails de cette conformité sont documentés dans [_bmad-output/planning-artifacts/research-swiss-co-958f.md](#) (Story 9-5-4 « Recherche réglementaire Swiss CO Art. 957a / 958f »).

Synthèse :

- al. 1 : enregistrement intégral, fidèle et systématique → garanti par la saisie partie double et l'`audit_log` immutable.
- al. 2 : principe de régularité → instancié par les patterns techniques Kesh (clarté multilingue, justification, traçabilité).
- al. 3 : pièce comptable papier ou électronique → support électronique autorisé.
- al. 4 : monnaie nationale ou monnaie principale → CHF par défaut, multi-monnaies prévu v0.2.
- al. 5 : langues nationales ou anglais → i18n FR/DE/IT/EN actif.

11.2 Code des obligations Art. 958f — Conservation 10 ans

- al. 1 : conservation 10 ans à compter de la fin de l'exercice → garanti par la persistance MariaDB (sous réserve d'admin DB + backup approprié).
- al. 2 : rapport de gestion imprimé et signé → **responsabilité de la PME utilisatrice**, hors scope logiciel.
- al. 3 : support libre + lien garanti + lisibilité possible → garanti par `audit_log` + SHA-256 + format CSV/PDF ouverts du ZIP d'export.

11.3 Ordonnance OLICo (RS 221.431)

- Art. 1 : grand livre + journal → implémentés Kesh.
- Art. 3 : intégrité (modification apparente) → garanti par `audit_log` insert-only.
- Art. 9 al. 1.b : supports modifiables (disque dur, SSD) → conditions 1-4 satisfaites (cf. section sécurité plus haut).
- Art. 10 : contrôle et migration → test de restauration annuel obligatoire (cf. section Sauvegarde).

11.4 LSCSE (signature électronique qualifiée)

La signature électronique qualifiée n'est **pas obligatoire** pour la conservation comptable PME selon l'OLICo Art. 9 (« p. ex. signature électronique »). Une implémentation QES est prévue en v0.2 (Epic 14, issue #98) pour les PME souhaitant un niveau supplémentaire de défense.

11.5 Loi sur la TVA (LTVA)

Kesh v0.1 ne couvre pas encore la TVA (prévue Epic 11). La conformité LTVA Art. 70 (factures électroniques) sera couverte dans cette future implémentation.

11.6 Documentation à conserver

Conformément à OLICo Art. 9 al. 1.b ch. 4 (« procédures et modes d'utilisation consignés »), conserver dans votre archive comptable :

- Une copie de ce manuel administrateur (version utilisée).
- Une copie du manuel utilisateur.
- Une copie du document de recherche réglementaire ([research-swiss-co-958f.md](#)).
- Les procès-verbaux de test de restauration annuels.
- Le procès-verbal de migration (cf. `metadata.json` des ZIP de souveraineté).

! Astuce

Une revue juridique externe (avocat suisse spécialisé en droit commercial / IT) est recommandée mais non bloquante pour une publication commerciale. Coût ordre-de-grandeur : CHF 1'000–3'000 pour une PME. Voir le document de recherche §Verdict pour les détails.

12 Dépannage

12.1 Kesh ne démarre pas

Symptômes Le container `kesh` sort immédiatement après `docker compose up`, ou `docker compose ps` montre `kesh` en `Restarting`.

```
docker compose logs kesh | tail -50
```

Causes courantes :

- **DATABASE_URL invalide** : vérifier le format `mysql://user:pass@host:port/db`.
- **DB pas encore prête** : ajouter un `depends_on` avec `condition: service_healthy` dans `docker-compose.yml`.
- **JWT_SECRET trop court** : doit faire au moins 64 caractères.
- **Migrations DB échouées** : voir log détaillé. Solution : restaurer depuis sauvegarde et investiguer.

12.2 Erreurs 401 en cascade côté frontend

Diagnostic

Symptômes Plusieurs appels API consécutifs échouent en `401 Unauthorized` alors que l'utilisateur est censé être authentifié.

Cause Refresh token expiré ou invalidé (cf. KF #54 « E2E test helpers cascade 401 », fermée Epic 9.5).

Solution L'utilisateur doit se reconnecter. Si le problème persiste pour tous les utilisateurs, vérifier l'horloge système (drift NTP).

12.3 Performance lente sur de gros datasets

Symptômes Génération de rapports ou exports ZIP > 30 secondes sur des companies avec > 50'000 écritures.

```
docker compose exec db mariadb -u root -p"${MARIADB_ROOT_PASSWORD}" -e \  
"SELECT COUNT(*) FROM kesh.journal_entries GROUP BY company_id;"
```

Solutions

- Augmenter `innodb_buffer_pool_size` (cf. config MariaDB).
- Vérifier que les index FULLTEXT sont en place (Story 7-4).
- Surveiller `slow.log` et optimiser les requêtes lentes éventuelles.
- Considérer une migration vers MariaDB 11.5+ qui inclut des optimisations supplémentaires.

12.4 Reset du mot de passe administrateur

Si l'administrateur a perdu son mot de passe et qu'il n'y a pas d'autre administrateur :

```
docker compose exec kesh kesh-cli admin reset-password \  
--email admin@example.ch \  
--new-password '<nouveau_mot_de_passe>'
```

L'opération est tracée dans `l'audit_log`.

12.5 Données corrompues détectées

Si une vérification d'intégrité (SHA-256 d'un export ZIP ne correspond plus, `audit_log` incohérent, etc.) révèle une corruption :

1. **Arrêter immédiatement** l'instance : `docker compose stop kesh`.
2. Conserver une copie de l'état actuel : `tar czf kesh-corrupted-$(date +%F).tar.gz /opt/kesh`.
3. Restaurer depuis la dernière sauvegarde saine (vérifier le SHA-256 de la sauvegarde elle-même avant restauration).
4. Analyser la cause racine : intrusion sécurité ? corruption disque ? bug applicatif ?
5. Documenter l'incident pour l'audit OLICo.

X Danger

Une corruption de données comptables peut avoir des implications légales sévères. Ne pas tenter de « réparer » manuellement les données corrompues — restaurer depuis

sauvegarde et investiguer hors production.

13 Maintenance

13.1 Rotation des logs

Configurer logrotate pour éviter que les logs Docker ne saturent le disque :

```
# /etc/docker/daemon.json
{
  "log-driver": "json-file",
  "log-opts": {
    "max-size": "10m",
    "max-file": "5"
  }
}
```

Puis redémarrer Docker : `sudo systemctl restart docker`.

13.2 Maintenance MariaDB

Tâches périodiques recommandées :

Tâche	Description	Fréquence
OPTIMIZE TABLE	Défragmentation des tables InnoDB.	Trimestrielle
ANALYZE TABLE	Mise à jour des statistiques d'optimiseur.	Mensuelle
Vérification cohérence	<code>mariadb-check --all-databases</code> .	Mensuelle
Purge binlog	Suppression des binary logs expirés (auto via <code>expire_logs_days</code>).	Quotidienne (auto)

13.3 Vérification périodique de l'intégrité (OLICo Art. 10)

Conformément à OLICo Art. 10 al. 1, vérifier l'intégrité des supports d'information régulièrement :

```
#!/bin/bash
```



```
# /usr/local/bin/kesh-integrity-check.sh
# Verification mensuelle de l'integrite des sauvegardes

BACKUP_DIR="/var/backups/kesh"
LOG_FILE="/var/log/kesh-integrity.log"

echo "  Verification integrite $(date +%F)  " >> "${LOG_FILE}"

cd "${BACKUP_DIR}"
for backup in kesh_*.sql.gz; do
    if sha256sum -c "${backup}.sha256" >> "${LOG_FILE}" 2>&1; then
        echo "  ${backup} OK" >> "${LOG_FILE}"
    else
        echo "  ${backup} INTEGRITE COMPROMISE - ALERTE" >> "${LOG_FILE}"
        # Envoyer une alerte (mail, Slack, PagerDuty, etc.)
    fi
done
```

14 Annexes

14.1 Liste complète des variables d'environnement

Voir section 5.1.

14.2 Commandes CLI utiles

```
# Statistiques globales d'une instance
docker compose exec kesh kesh-cli stats

# Liste des companies
docker compose exec kesh kesh-cli company list

# Forcer la regeneration des QR Bills d'une company
docker compose exec kesh kesh-cli invoice regenerate-qr --company-id <ID>

# Forcer la re-execution des migrations DB
docker compose exec kesh kesh-cli migrate

# Export complet de la base au format SQL (alternative a mariadb-dump)
docker compose exec kesh kesh-cli export --company-id <ID> --format sql
```

```
# Verification de l'integrite d'un export ZIP (verification SHA-256)
docker compose exec kesh kesh-cli verify --export-id <UUID>
```

14.3 Référence des ports

Service	Port	Notes
Frontend HTTPS (utilisateur)	443	Via reverse proxy
API Axum (interne)	80	Localhost uniquement
MariaDB	3306	Localhost ou réseau privé
Prometheus metrics (optionnel)	9090	Réseau interne monitoring

14.4 Glossaire

Audit-trail Trace immuable de toutes les actions métier (table `audit_log`).

CAMT.053 Standard ISO 20022 d'extrait de compte bancaire suisse.

Company Entité comptable (PME, association) dans Kesh ; isolation multi-tenant par `company_id`.

Fiscal year Exercice comptable, généralement annuel (1^{er} janvier – 31 décembre par défaut).

IDOR Insecure Direct Object Reference, classe d'attaque évitée via le scoping `company_id`.

Journal entry Écriture comptable en partie double.

OLICo Ordonnance sur la tenue et la conservation des livres de comptes (RS 221.431).

pain.001 Standard ISO 20022 d'ordre de paiement (Customer Credit Transfer Initiation).

QES Qualified Electronic Signature (LSCSE Art. 2 let. e).

QR Bill Standard suisse 2020+ de facture avec QR code (SIX).

RBAC Role-Based Access Control.

Refresh token Token longue durée pour renouveler un access token JWT.

Tenant Synonyme de *company* dans un contexte multi-tenant.

14.5 Liens externes

- [Dépôt GitHub Kesh](#)
- [Code des obligations RS 220](#)
- [Ordonnance OLICo RS 221.431](#)
- [Loi LSCSE RS 943.03](#)
- [Guide PME comptabilité \(SECO\)](#)

- QR Bill SIX
- ISO 20022 (CAMT.053, pain.001)

14.6 Support et communauté

- Issues GitHub : <https://github.com/guycorbaz/kesh/issues>
- Discussions : <https://github.com/guycorbaz/kesh/discussions>
- Documentation technique : <https://github.com/guycorbaz/kesh/tree/main/docs>

i Note**À propos de cette version**

Ce document décrit Kesh version **0.1.0-dev** (cible release : v0.1, publié le 2026-05-20). Les manuels Kesh sont mis à jour à chaque release. Pour la dernière version, consultez <https://github.com/guycorbaz/kesh>.

Si vous identifiez une incohérence entre ce manuel et le comportement effectif du logiciel, merci d'ouvrir une issue sur GitHub avec le template `bug_report.yml`.